

Zusammenfassung der wichtigsten Anforderungen

Bereich	Kern-Need
Text-Kernel	Import gängiger Formate (PDF, DOCX ...). Automatische Struktur-Analyse (Szenen, Rollen, Seiten) und Ablage in einer internen Text-DB.
Kollaboration	Alle Nutzer*innen sehen immer dieselbe, live synchronisierte Fassung; Offline-Bearbeitung mit späterem Push. Granulares Rechte-Modell (Regie = Admin, Regieassistent = Maintainer, Gewerke/Schauspiel = Kommentare etc.). Git-ähnliche, aber GUI-freundliche Versionsverwaltung.
Audio-Mitlesen	Toggle-Modus, der Live-Mikrofon abhört, erkannte Passagen dem Skript zuordnet, Satz/Wort hervorhebt und scrollt. Optionale Fein-Tuning-Modelle pro Produktion.
Gewerke-Integration	Notizen & Cue-Listen; perspektivisch Trigger (OSC/MIDI), Import/Export zu Licht-/Tonsystemen.
Sicherheit	EU-Hosting, optionale On-Prem-Instanz, Ende-zu-Ende-Verschlüsselung für Text, Audio & Medien, 2-Faktor-Auth; Zusatz-Module für höhere Compliance.
Erweiterungen	Chat/Threads, Kalender & Callsheets, Medien-Ablage verknüpft mit Textstellen, Dramaturgie-Kommentare.

Dein MVP – „Version 1.0“

Priorität	Funktion	Kurzbeschreibung
P1	Text-Import & -Parser	Wizard: Upload PDF/DOCX → Extrahiert Szenen/Rollen/Seiten, legt verlinkte Objekte in DB an.
P1	Edit-Interface (Web-App)	Rich-Text-Editor mit Theater-Layout (Seitenzahl, Rollenfarbe). Zeigt Live-Cursor anderer Nutzer*innen, offline-fähig durch lokal geklonte Datei + CRDT-Sync.
P1	Live-Sync & Conflict-Handling	Automatisches Speichern, Optimistic Locking mit visueller Merge-Hilfe („Änderungen übernehmen / verwerfen“) – Git-Log im Hintergrund, GUI-Simplifizierung („Regieassistent*in löst Konflikte“).

P1	Rechtesystem	Rollenverwaltung, pro Rolle: <i>lesen / kommentieren / vorschlagen / mergen / administrieren</i> .
P2	Kommentar-Layer	Inline-Comments, filterbar nach Abteilung (Schauspiel, Licht ...).
P2	Grund-Audio-Mitlesen (Passiv-Sync)	Deutsch (Standard) + Englisch. Erkennt gesamte Zeilen, springt zum richtigen Absatz.
P3	Export zurück in Ursprungsformat	PDF & DOCX-Export in Layout-Treue mit eingeflossenem Stand.
P3	Benachrichtigungen	E-Mail/Push bei Text-Änderungen oder neuen Kommentaren.

P1 = Must-have für produktiven Probenstart

P2 = "Nice, aber MVP+"

P3 = Folgerelease (sobald Kern stabil)

Grober Architektur-Vorschlag

1. Frontend

- Web-App (React + Service-Worker für Offline).
- Tailwind UI; CRDT-Engine (z. B. Automerge, Yjs) für konfliktfreie Live-Edits.

2. Backend

- **API-Layer** (Node/Deno oder Go) mit GraphQL oder REST.
- **Text-Store**: PostgreSQL + JSONB; Git-ähnliche Commits als Objekte (Parents, Diffs).
- **Realtime**: WebSocket-Hub (e.g. NATS/Socket.IO).

3. Audio-Service

- Separate Microservice (Python) mit Streaming-ASR (OpenAI Whisper-basierend, in-house feinetunt).
- Federated: Edge-Device kann Offline-ASR nutzen, Server schickt Align-Daten zurück.

4. Security

- TLS-only, E2E-Crypto (libsodium) für Text-payloads; Audio optional E2E.
- Auth via OAuth2 / SAML (Theater-SSO) + WebAuthn 2FA.
- Deploy wahlweise: -Cloud (Hetzner, OVH, Exoscale) oder Container-Bundle für On-Prem K8s.

Offene Punkte & Risiken

Thema	Frage	Impact
Layout-Treue	Reicht es, Seitenumbrüche heuristisch zu rekonstruieren oder brauchst du 100 % gleiche Zeilenumbrüche wie Original-PDF?	Entscheidet über einfachen Parser vs. komplexen PDF-Layout-Engine.
ASR-Qualität	Wie viele Produktions-Sprachen brauchst du im ersten Jahr?	Trainings-Budget & Model-Hosting-Kosten.
Cue-Trigger	Reine Notiz vs. echtes OSC/MIDI-Out; wartet Integrationen?	Entwicklungsaufwand steigt stark, wenn Hardware-Tests nötig.